

Báo cáo đồ án cuối khóa

Python cho Khoa học dữ liệu / Dự đoán calories

Mục lục

1	Giới thiệu bài toán và dữ liệu	2
1.1	Mô tả dữ liệu và EDA	2
1.2	Phân tích dữ liệu	2
1.3	Ý nghĩa thực tiễn của bài toán	5
2	Giới thiệu các Class và phương thức trong project	5
2.1	Package: src.data	5
2.1.1	src/data/io.py	6
2.1.2	src/data/transformer.py	6
2.1.3	src/data/preprocessor.py	7
2.2	Package: src.models	8
2.2.1	src/models/evaluator.py	8
2.2.2	src/models/io.py	9
2.2.3	src/models/trainer.py	9
2.2.4	src/models/optimizer.py	10
2.3	Package: src.visualization	10
2.3.1	src/visualization/eda.py	10
2.3.2	src/visualization/model_plots.py	11
3	Tổng quan thư viện và phương pháp tối ưu siêu tham số	11
3.1	Thư viện học máy sử dụng	11
3.2	Các mô hình được sử dụng	12
3.3	Chỉ số đánh giá mô hình	12
3.4	Tối ưu siêu tham số bằng phương pháp Bayesian	12
4	Kết quả thực nghiệm	12
4.1	So sánh các mô hình học máy	13
4.2	Phân tích đặc trưng quan trọng với mô hình Lasso	13
5	Bảng công việc	14

1 Giới thiệu bài toán và dữ liệu

Trong project này, bài toán được lựa chọn là **hồi quy (regression)**, với mục tiêu dự đoán **lượng Calories của các món ăn nhanh** tại những chuỗi cửa hàng lớn, dựa trên các đặc trưng dinh dưỡng của từng sản phẩm.

Bộ dữ liệu sử dụng là *Fast_Food_Dataset*, được công bố trên Kaggle¹.

1.1 Mô tả dữ liệu và EDA

Dưới đây là mô tả chi tiết các trường dữ liệu có trong bộ dữ liệu, được thể hiện trong Bảng 1.

Cột	Mô tả
Company	Tên công ty hoặc thương hiệu cung cấp sản phẩm.
Item	Tên món ăn hoặc sản phẩm cụ thể.
Calories	Tổng lượng calo trong một khẩu phần.
Calories from Fat	Lượng calo có nguồn gốc từ chất béo.
Total Fat (g)	Tổng lượng chất béo (gram) trong một khẩu phần.
Saturated Fat (g)	Lượng chất béo bão hòa (gram).
Trans Fat (g)	Lượng chất béo chuyển hóa (gram).
Cholesterol (mg)	Hàm lượng cholesterol (mg).
Sodium (mg)	Hàm lượng natri/muối (mg).
Carbs (g)	Tổng lượng carbohydrate (gram).
Fiber (g)	Hàm lượng chất xơ (gram).
Sugars (g)	Lượng đường (gram).
Protein (g)	Hàm lượng protein (gram).
Weight Watchers Pnts	Điểm số theo hệ thống đánh giá dinh dưỡng Weight Watchers.

Bảng 1: Bảng mô tả dữ liệu *Fast_Food_Dataset*

Hình 1 thể hiện phân phối của các đặc trưng (features) trước xử lý, cho thấy dữ liệu có độ lệch khá lớn ở nhiều thuộc tính dinh dưỡng.

Hình 2 cho thấy phân bố các giá trị khuyết (missing values), trong đó **Calories from Fat** là đặc trưng có tỉ lệ thiếu nhiều nhất.

Biểu đồ hộp của các đặc trưng quan trọng được thể hiện ở Hình 3, cho thấy bộ dữ liệu chứa khá nhiều giá trị ngoại lai (outliers).

Cuối cùng, Hình 4 trình bày ma trận tương quan giữa các đặc trưng dinh dưỡng.

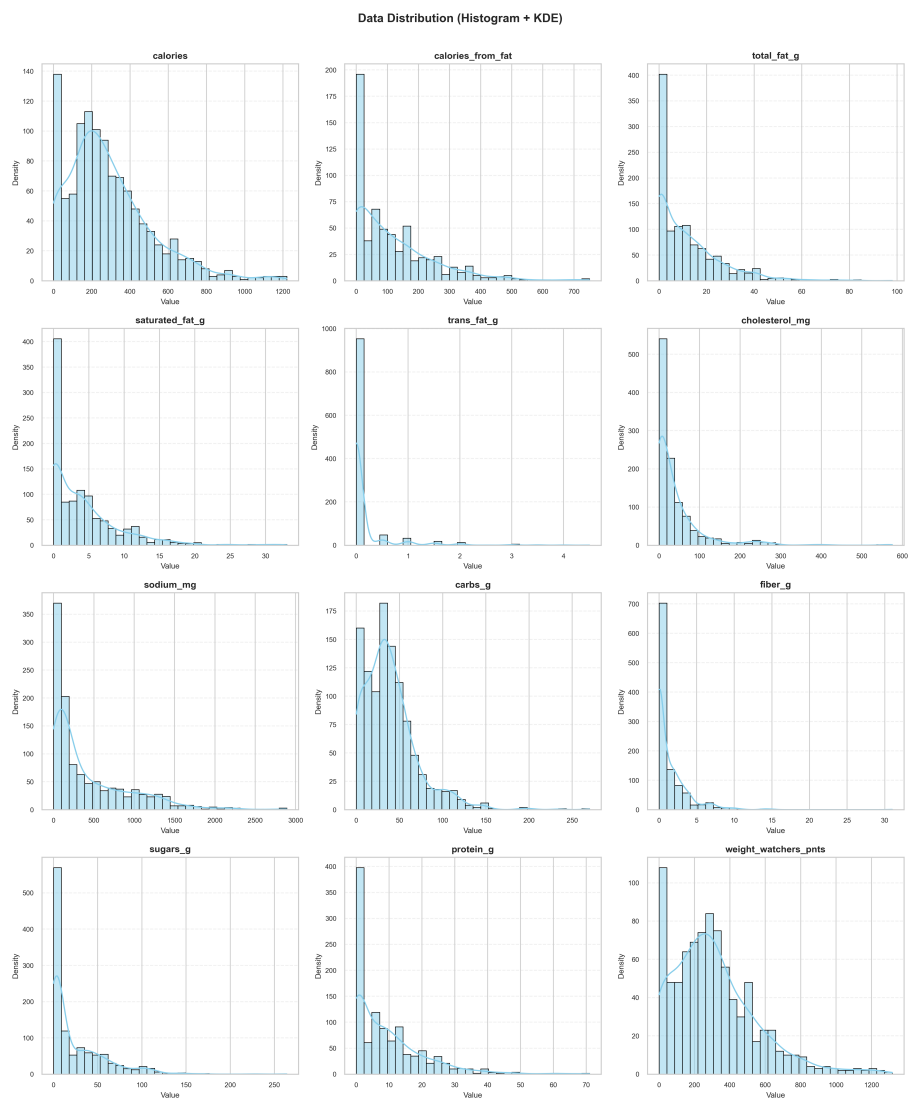
1.2 Phân tích dữ liệu

Dựa trên heatmap tương quan ở Hình 4, nhóm thu được một số nhận xét quan trọng:

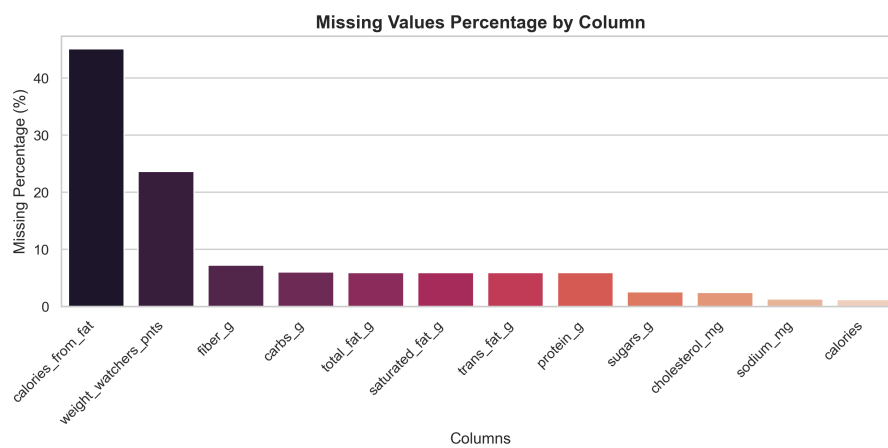
- **Calories from Fat** và **Total Fat (g)** có tương quan gần như tuyệt đối ≈ 1 Điều này hợp lý về mặt dinh dưỡng:

$$\text{Calories from Fat} \approx \text{Total Fat (g)} \times 9,$$

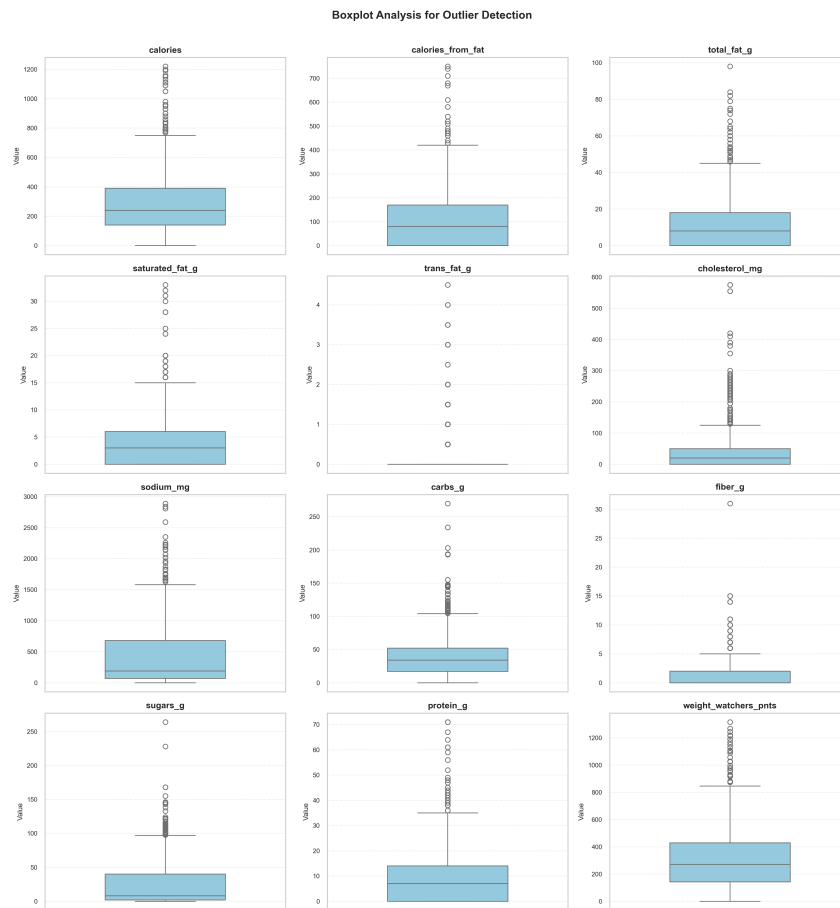
¹<https://www.kaggle.com/datasets/tan5577/nutritonal-fast-food-dataset>



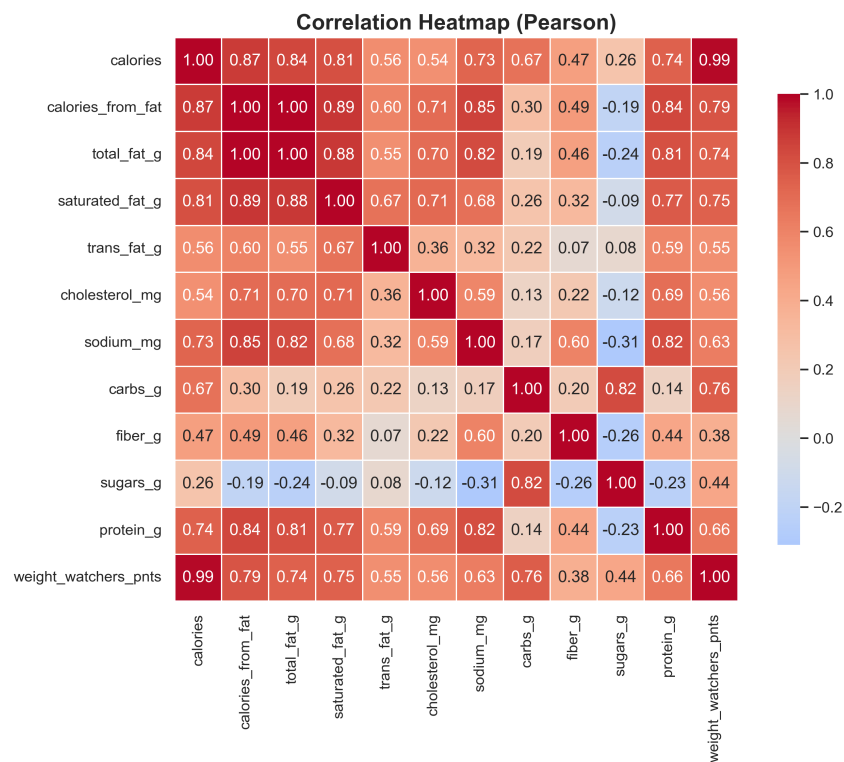
Hình 1: Phân phối dữ liệu các đặc trưng



Hình 2: Biểu đồ giá trị khuyết trong dữ liệu



Hình 3: Biểu đồ hộp (boxplot) các đặc trưng



Hình 4: Heatmap tương quan các đặc trưng

nên hai cột này chứa cùng thông tin, chỉ khác đơn vị. Thêm vào đó, **Calories from Fat** có nhiều missing values nhất, nên nhóm **loại bỏ Calories from Fat**.

- **Weight Watchers Pnts** có tương quan rất cao với **Calories**, vì bản chất nó được tính dựa trên lượng calo. Giữ lại đặc trưng này sẽ gây **rò rỉ dữ liệu (data leakage)**, do mô hình sẽ gián tiếp nhìn thấy biến mục tiêu. Vì vậy nhóm **loại bỏ Weight Watchers Pnts**.
- **Company** và **Item** chỉ mang tính mô tả tên chuỗi cửa hàng và tên món ăn, không chứa thông tin dinh dưỡng. Việc mã hóa các cột này sẽ làm tăng số chiều không cần thiết, nên nhóm quyết định **loại bỏ cả hai**.

Tóm lại, các cột bị loại bỏ gồm:

Calories from Fat, Weight Watchers Pnts, Company, Item

Ngoài ra, dựa trên ngữ nghĩa dinh dưỡng của dữ liệu, nhóm quyết định **chuyển toàn bộ giá trị âm thành giá trị dương** (do đây khả năng là lỗi nhập liệu—typo), thay vì loại bỏ danh sách mẫu.

Dựa trên Hình 3, dữ liệu tồn tại khá nhiều ngoại lệ, do đó cần có bước xử lý ngoại lai trước khi huấn luyện mô hình.

1.3 Ý nghĩa thực tiễn của bài toán

- **Hỗ trợ người dùng kiểm soát dinh dưỡng:**
 - Ước lượng nhanh lượng calories trong bữa ăn.
 - Lựa chọn món ăn phù hợp với chế độ ăn kiêng hoặc nhu cầu tập luyện.
 - Ra quyết định tốt hơn khi buộc phải chọn các sản phẩm thức ăn nhanh.
- **Hỗ trợ chuỗi cửa hàng và doanh nghiệp thực phẩm:**
 - Ước lượng calories cho món mới khi thay đổi công thức.
 - Tối ưu menu để có nhiều lựa chọn “healthy”.

Nhìn chung, mô hình dự đoán lượng calories không chỉ là một bài toán hồi quy đơn thuần mà còn là một **công cụ hỗ trợ ra quyết định** hữu ích cho người tiêu dùng và doanh nghiệp trong ngành thực phẩm.

2 Giới thiệu các Class và phương thức trong project

Phần này mô tả cấu trúc mã nguồn của project và giải thích **chức năng của từng Class cùng các phương thức (methods) chính**. Hệ thống được tổ chức trong thư mục **src/** và chia thành ba package chính: **src.data**, **src.models** và **src.visualization**. Cách trình bày được thiết kế theo phong cách tài liệu kỹ thuật, giúp người đọc dễ dàng tra cứu khi muốn xem lại luồng xử lý của chương trình.

2.1 Package: **src.data**

Package này chịu trách nhiệm xử lý dữ liệu đầu vào, bao gồm: đọc/ghi file, làm sạch dữ liệu thô, xử lý giá trị khuyết, ngoại lai và chuẩn hoá dữ liệu để đưa vào mô hình học máy.

2.1.1 src/data/io.py

Class: DataIO

Class chuyên trách việc Đọc/Ghi file và chuẩn hóa tên cột. Đảm bảo dữ liệu được nạp vào và lưu ra đúng định dạng, nhất quán với toàn bộ pipeline.

Methods:

`load_data(filepath)`

Nạp dữ liệu từ các file (CSV, XLSX, JSON) vào `pandas DataFrame`. Hỗ trợ xử lý lỗi khi file không tồn tại hoặc sai định dạng, giúp chương trình không bị crash đột ngột.

`save_data(data, filepath)`

Lưu `DataFrame` hiện tại vào file CSV tại đường dẫn chỉ định. Được dùng sau khi dữ liệu đã qua xử lý để lưu lại phiên bản *sạch*.

`clean_column_names(data)`

Chuẩn hóa tên cột: loại bỏ ký tự đặc biệt, khoảng trắng thừa và chuyển về dạng `snake_case` để thuận tiện cho việc code và tương thích với các hàm xử lý tiếp theo.

2.1.2 src/data/transformer.py

Class: DataTransformer

Class chứa các thuật toán cốt lõi để xử lý dữ liệu như: xử lý giá trị thiếu, xử lý ngoại lai, chuẩn hóa, mã hóa, và chuyển đổi kiểu dữ liệu. `DataTransformer` hoạt động ở dạng các hàm thuần (pure functions) trên `DataFrame`.

Methods:

`auto_detect_columns(data)`

Tự động phát hiện và phân loại các cột theo nhóm: *numeric*, *categorical*, *datetime*. Thông tin này được dùng để quyết định chiến lược tiền xử lý phù hợp cho từng loại cột.

`convert_columns_to_numeric(data, columns)`

Ép kiểu các cột được chỉ định thành kiểu số (int/float), xử lý các ký tự không hợp lệ (ví dụ: dấu phẩy, ký hiệu đơn vị...).

`auto_convert_numeric_columns(data, threshold)`

Tự động rà soát các cột kiểu `object` và cố gắng chuyển sang kiểu số. Nếu tỉ lệ giá trị chuyển đổi thành công vượt `threshold`, cột đó sẽ được coi là cột số.

`convert_to_datetime(data, columns, date_format)`

Chuyển đổi các cột được chỉ định sang kiểu `datetime` theo định dạng ngày tháng cung cấp, hỗ trợ chuẩn hóa dữ liệu thời gian.

`clean_negative_values(data)`

Thay thế tất cả giá trị âm trong các cột số bằng giá trị tuyệt đối của chúng.

Điều này bám sát quyết định xử lý ở Mục 1, vì các chỉ số dinh dưỡng không nên mang giá trị âm.

`handle_missing_values(data, strategies, ...)`

Xử lý giá trị thiếu theo chiến lược cấu hình (điền bằng `mean`, `median`, `mode` hoặc `drop`). Cho phép cấu hình khác nhau cho từng cột hoặc từng nhóm cột.

`handle_outliers(data, method, ...)`

Phát hiện và xử lý các giá trị ngoại lai trong cột số sử dụng các phương pháp như `IQR`, `Z-score` hoặc `Isolation Forest`, phù hợp với phần mô tả ở Hình 3.

`encode_categorical(data, strategy)`

Mã hóa các biến phân loại thành dạng số để mô hình có thể sử dụng (hỗ trợ `Label Encoding` và `One-Hot Encoding`).

`scale_features(data, strategy, ...)`

Chuẩn hóa dữ liệu số về cùng một khoảng (sử dụng `StandardScaler` hoặc `RobustScaler`), khớp với các chiến lược scaling đã trình bày trong Bảng 2.

`remove_duplicates(data, subset)`

Kiểm tra và loại bỏ các hàng trùng lặp hoàn toàn hoặc trùng trên một tập cột con (`subset`), giúp giảm nhiễu.

`drop_features(data, features)`

Loại bỏ hoàn toàn các cột không cần thiết hoặc đã được xác định là gây ra *data leakage*.

2.1.3 src/data/preprocessor.py

Class: DataPreprocessor

Class đóng vai trò *orchestrator* (bộ điều phối), kết hợp `DataIO` và `DataTransformer` để tạo thành một pipeline xử lý dữ liệu hoàn chỉnh. Class này lưu giữ trạng thái của dữ liệu trong quá trình biến đổi.

Methods:

`__init__(...)`

Khởi tạo đối tượng với các tham số cấu hình cho toàn bộ quy trình (chiến lược xử lý missing values, scaling, outlier, encoding...).

`load_data(filepath, ...)`

Gọi `DataIO.load_data()` để nạp dữ liệu từ file và thực hiện ngay các bước chuẩn hóa sơ bộ như chuẩn hóa tên cột, ép kiểu cơ bản.

`save_data(filepath)`

Lưu trạng thái dữ liệu hiện tại (sau khi xử lý) xuống file, giúp tái sử dụng hoặc chia sẻ cho các bước phân tích khác.

`auto_detect_columns()`

Phân loại các cột (numeric, categorical, datetime) và lưu thông tin này vào thuộc

tính nội bộ của class.

`convert_to_datetime(...)`

Gọi hàm chuyển đổi ngày tháng và cập nhật lại `DataFrame` nội bộ.

`clean_negative_values()`

Thực hiện làm sạch giá trị âm trên dữ liệu hiện có, đảm bảo tính hợp lý về mặt ngữ nghĩa dinh dưỡng.

`handle_missing_values(...)`

Thực hiện xử lý giá trị thiếu. Hỗ trợ chế độ `fit` (học tham số điền trên train set) và `transform` (áp dụng cùng tham số đó cho test set) để tránh *data leakage*.

`handle_outliers(...)`

Loại bỏ hoặc cắt ngưỡng các giá trị ngoại lai theo cấu hình được chỉ định.

`encode_categorical(...)`

Mã hóa biến phân loại và lưu trữ các encoder để áp dụng nhất quán cho dữ liệu mới.

`scale_features(...)`

Chuẩn hóa dữ liệu số, lưu lại `scaler` đã `fit` để dùng cho tập kiểm tra hoặc dữ liệu triển khai thực tế.

`remove_duplicates(...)`

Loại bỏ các hàng trùng lặp trên dữ liệu hiện tại.

`drop_features(...)`

Xóa các cột không còn cần thiết (ví dụ: `Calories from Fat`, `Weight Watchers Pnts`, `Company`, `Item` như phân tích ở mục trước).

`get_processed_data()`

Trả về `DataFrame` kết quả sau khi đã trải qua toàn bộ pipeline tiền xử lý.

2.2 Package: `src.models`

Package này quản lý vòng đời của mô hình học máy: từ huấn luyện, tối ưu siêu tham số cho đến đánh giá và lưu mô hình.

2.2.1 `src/models/evaluator.py`

Class: `ModelEvaluator`

Class tiện ích (utility) cung cấp các phương thức tính để tính toán các chỉ số đánh giá hiệu quả mô hình.

Methods:

`evaluate_model(model, X_test, y_test, model_name)`

Dự đoán trên tập test và tính toán các metrics: `MSE`, `RMSE`, `MAE`, R^2 . Kết quả được trả về dưới dạng `dict` để dễ lưu trữ và phân tích.

```
get_feature_importance(model, feature_names, ...)
```

Trích xuất độ quan trọng của các đặc trưng từ mô hình đã huấn luyện (hỗ trợ cả mô hình tuyến tính và tree-based). Kết quả này liên kết trực tiếp với phần phân tích ở Hình 6.

2.2.2 src/models/io.py

Class: ModelIO

Class chịu trách nhiệm *serialization* (lưu trữ) và *deserialization* (khôi phục) các mô hình và kết quả đánh giá.

Methods:

```
load_model(filepath, method)
```

Đọc file mô hình từ ổ cứng và khôi phục thành object Python. Hỗ trợ các phương thức như `joblib` và `pickle`.

```
save_model(model, model_name, ...)
```

Lưu object mô hình xuống file để tái sử dụng sau này, có thể tự động gắn thêm `timestamp` nếu không chỉ định tên file cụ thể.

```
save_results(results, filepath, format)
```

Xuất kết quả đánh giá (metrics) ra file CSV hoặc JSON, giúp liên kết với file Google Sheets trong Mục 4.

2.2.3 src/models/trainer.py

Class: ModelTrainer

Class trung tâm quản lý quy trình huấn luyện, kết nối giữa dữ liệu đã xử lý, các thuật toán mô hình và bước tối ưu siêu tham số.

Methods:

```
__init__(random_state)
```

Khởi tạo Trainer với `random_state` cố định để đảm bảo tính tái lập (*reproducibility*).

```
load_data(data, target_column)
```

Nhận dữ liệu đã tiền xử lý và xác định cột mục tiêu `target_column` (trong project là `Calories`).

```
split_data(test_size, stratify)
```

Chia dữ liệu thành tập Train/Test theo tỉ lệ chỉ định, hỗ trợ `stratify` nếu cần.

```
set_training_data(train_processed, test_processed, ...)
```

Thiết lập dữ liệu đầu vào (X) và nhãn (y) cho quá trình huấn luyện từ các `DataFrame` đã xử lý.

```
initialize_models()
```

Khởi tạo danh sách các mô hình sẽ dùng: `LinearRegression`, `Ridge`, `Lasso`,

ElasticNet, DecisionTree/RandomForest, LightGBM...

`train_models(models_to_train)`

Thực hiện vòng lặp `fit` cho các mô hình được yêu cầu.

`optimize_params(model_name, ...)`

Sử dụng Bayesian Optimization (thông qua Optuna) để tìm bộ siêu tham số tốt nhất cho một mô hình cụ thể.

`evaluate_models()`

Đánh giá toàn bộ các mô hình đã huấn luyện trên tập Test và tìm ra mô hình tốt nhất dựa trên R^2 (kết quả liên quan tới Hình 5).

`get_feature_importance(model_name, top_n)`

Lấy danh sách các đặc trưng quan trọng nhất ảnh hưởng đến kết quả dự đoán của mô hình đó.

2.2.4 src/models/optimizer.py

Class: BayesianOptimizer

Class chuyên biệt thực hiện tối ưu hóa siêu tham số sử dụng thư viện Optuna.

Methods:

`__init__(X_train, y_train, ...)`

Khởi tạo Optimizer với dữ liệu huấn luyện và cấu hình Cross-Validation.

`optimize(model_name, n_trials)`

Định nghĩa không gian tìm kiếm (search space) cho từng loại model và chạy quá trình tối ưu hóa để tối đa hóa R^2 trên tập validation.

2.3 Package: src.visualization

Package này cung cấp các công cụ trực quan hóa giúp hiểu rõ dữ liệu và kết quả mô hình thông qua các biểu đồ, gắn với các hình đã trình bày ở phần trước.

2.3.1 src/visualization/eda.py

Class: EDA

Class thực hiện *Exploratory Data Analysis* (Phân tích Dữ liệu Khám phá), giúp có cái nhìn tổng quan về dữ liệu trước khi mô hình hóa.

Methods:

`__init__(data, show_plots)`

Khởi tạo với dữ liệu cần phân tích và cờ `show_plots` để quyết định có hiển thị trực tiếp biểu đồ hay chỉ lưu file.

`summary_statistics(save_path)`

Tính toán và hiển thị các thống kê mô tả, kiểm tra giá trị thiếu, trùng lặp và

độ lệch (skewness) của dữ liệu.

`correlation_analysis(method, save_path)`

Tính toán ma trận tương quan (Pearson, Spearman...) và vẽ Heatmap, tương ứng với Hình 4.

`data_distribution(save_path)`

Vẽ biểu đồ histogram kết hợp đường cong mật độ (KDE) để xem phân phối của từng biến số, như Hình 1.

`boxplot_analysis(save_path)`

Vẽ boxplot cho các đặc trưng số nhằm trực quan hóa ngoại lệ, như Hình 3.

`perform_eda(corr_method, save_path)`

Phương thức tiện ích chạy toàn bộ quy trình EDA theo trình tự chuẩn: thống kê mô tả, phân phối, tương quan, boxplot...

2.3.2 src/visualization/model_plots.py

Class: ModelVisualizer

Class chuyên biệt để trực quan hóa hiệu năng của các mô hình sau khi huấn luyện.

Methods:

`__init__(evaluation_results)`

Khởi tạo với kết quả đánh giá thu được từ `ModelTrainer` (ví dụ: bảng metrics của từng mô hình).

`plot_model_comparison(save_path)`

Vẽ biểu đồ cột so sánh các chỉ số MSE, RMSE, MAE, R^2 giữa các mô hình, tương ứng với Hình 5.

`plot_feature_importance(importance_df, save_path)`

Vẽ biểu đồ thanh ngang thể hiện mức độ đóng góp của từng đặc trưng vào quyết định của mô hình, tương ứng với Hình 6.

3 Tổng quan thư viện và phương pháp tối ưu siêu tham số

3.1 Thư viện học máy sử dụng

Trong nghiên cứu này, ba thư viện chính được sử dụng gồm **Scikit-learn (sklearn)**, **LightGBM** và **Optuna**.

Sklearn là thư viện phổ biến trong Python, cung cấp đầy đủ các thuật toán học máy truyền thống như hồi quy tuyến tính, cây quyết định và các phương pháp đánh giá mô hình.

LightGBM là thư viện chuyên dụng cho các mô hình cây tăng cường (Gradient Boosting), có tốc độ huấn luyện nhanh và hiệu quả cao với tập dữ liệu lớn.

Optuna là thư viện tối ưu siêu tham số tự động, hỗ trợ nhiều chiến lược tìm kiếm tiên tiến.

3.2 Các mô hình được sử dụng

Các mô hình hồi quy được áp dụng trong nghiên cứu gồm: **Linear Regression**, **Ridge Regression**, **Lasso Regression**, **Elastic Net**, **Decision Tree Regression** và **LightGBM Regression**. Nhóm mô hình tuyến tính giúp phân tích mối quan hệ tuyến tính giữa biến đầu vào và đầu ra, trong khi Decision Tree và LightGBM có khả năng mô hình hóa quan hệ phi tuyến phức tạp.

3.3 Chỉ số đánh giá mô hình

Hiệu quả của các mô hình được đánh giá thông qua ba chỉ số chính: **R-squared (R^2)**, **Mean Squared Error (MSE)** và **Mean Absolute Error (MAE)**. Chỉ số R^2 phản ánh mức độ giải thích của mô hình đối với dữ liệu, trong khi MSE và MAE đo lường sai số dự đoán giữa giá trị thực và giá trị dự đoán.

3.4 Tối ưu siêu tham số bằng phương pháp Bayesian

Quá trình tối ưu siêu tham số được thực hiện bằng phương pháp **Bayesian Optimization** thông qua thư viện Optuna. Phương pháp này xây dựng một mô hình xác suất cho hàm mục tiêu và lựa chọn bộ siêu tham số tối ưu dựa trên chiến lược cân bằng giữa khai thác và khám phá, giúp giảm đáng kể số lần thử nghiệm so với tìm kiếm lưới (Grid Search).

4 Kết quả thực nghiệm

Kết quả thực nghiệm chi tiết được tổng hợp trong file Google Sheets². Cụ thể:

- **Sheet 1:** Giá trị trung bình các metrics cho từng cấu hình tiền xử lý và mô hình.
- **Sheet 2:** Tên mô hình cho ra kết quả tốt nhất tương ứng với từng phương pháp xử lý dữ liệu.
- **Sheet 3:** Toàn bộ kết quả của tất cả các lần chạy, phục vụ cho việc đối chiếu và phân tích chi tiết.

Dựa trên kết quả trung bình các metrics, nhóm tiến hành xếp hạng các cấu hình tiền xử lý và thu được **top 5** phương pháp có kết quả trung bình tốt nhất như trong Bảng 2.

num_strategy	scaling_strategy	outlier_method	r2_score
drop	standard	zscore	0.9695
drop	robust	zscore	0.9638
mode	standard	zscore	0.9564
median	standard	zscore	0.9530
drop	standard	isolation_forest	0.9513

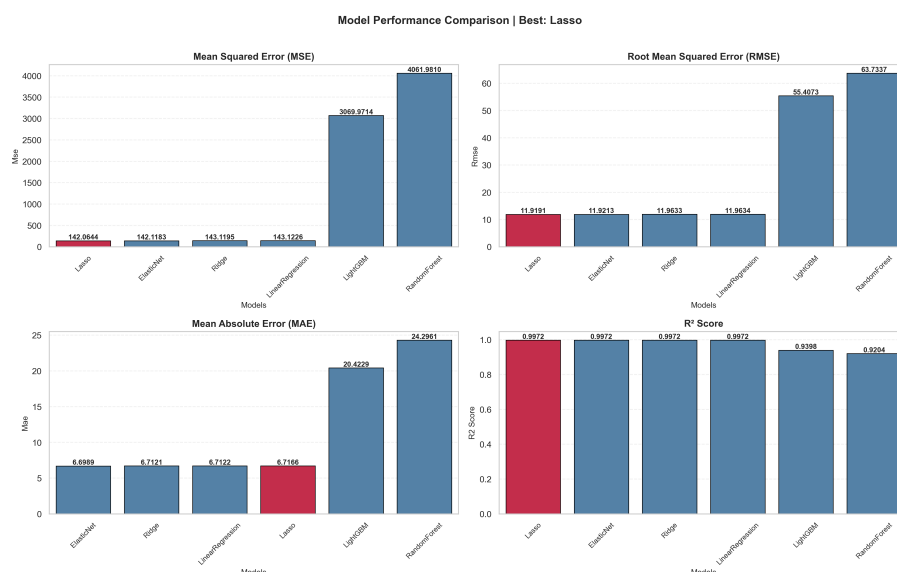
Bảng 2: Top 5 cấu hình tiền xử lý dữ liệu có kết quả trung bình tốt nhất

²<https://docs.google.com/spreadsheets/d/1DugDKIYezPlDGX1fH01LJ1cdBhQisnCArAE03LLE0dM/edit?usp=sharing>

Có thể thấy, các cấu hình sử dụng kết hợp **standard scaling** và phát hiện ngoại lệ bằng **zscore** thường xuyên xuất hiện trong nhóm tốt nhất, trong đó phương án **drop, standard, zscore** cho kết quả ổn định và nổi trội. Ngoài ra sau khi đã hoàn thành huấn luyện tất cả các phương pháp, ta thấy trong bài toán này các mô hình họ cây có kết quả không tốt bằng các mô hình tuyến tính. **Do đó, ở các bước tiếp theo, nhóm lựa chọn cấu hình tiền xử lý gồm: loại bỏ giá trị khuyết (drop), chuẩn hoá standard, và xử lý ngoại lệ bằng zscore để tiến hành tối ưu siêu tham số cho mô hình.**

4.1 So sánh các mô hình học máy

Sau khi cố định quy trình tiền xử lý như trên, nhóm tiến hành tối ưu siêu tham số, huấn luyện và đánh giá **6 mô hình** khác nhau. Kết quả so sánh các mô hình dựa trên các metrics đánh giá được biểu diễn trong Hình 5.



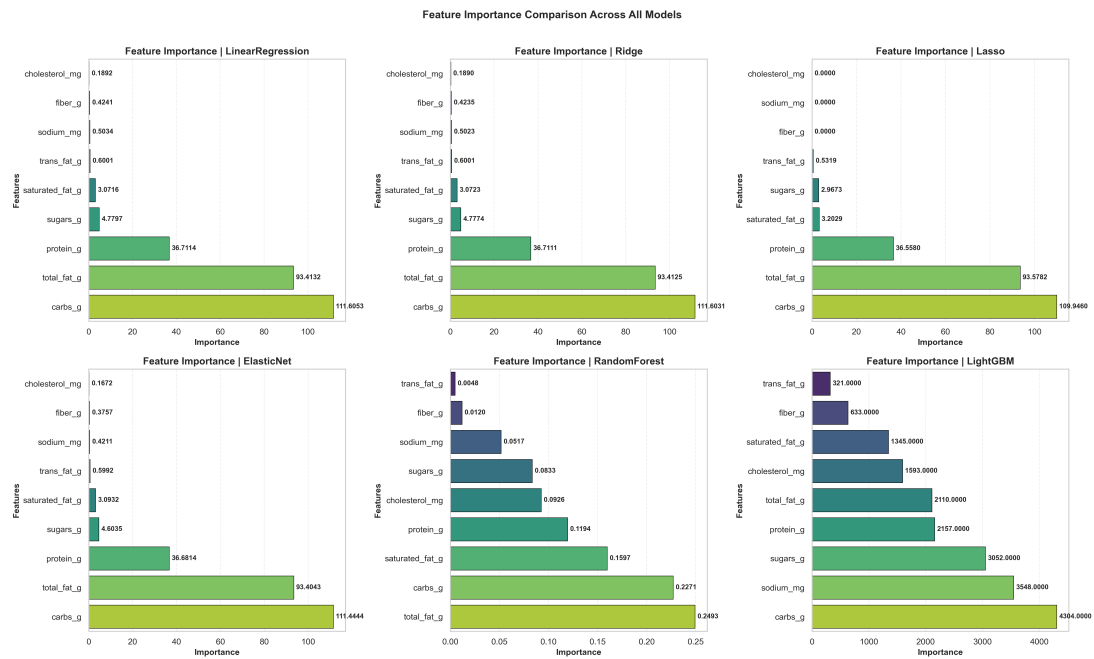
Hình 5: Kết quả so sánh 6 mô hình học máy

Quan sát Hình 5, có thể thấy các mô hình phát triển từ *linear regression* đều cho giá trị R^2 gần như tương đương nhau, cùng đạt khoảng **0.9972**. Sự sai khác ở 3 metrics còn lại (MAE, MSE, RMSE) là tương đối nhỏ và không mang ý nghĩa thực tiễn quá lớn trong bối cảnh bài toán, do đó có thể bỏ qua.

Dựa trên nhận định này, nhóm quyết định lựa chọn mô hình **Lasso** làm mô hình chính để phân tích sâu hơn.

4.2 Phân tích đặc trưng quan trọng với mô hình Lasso

Sau khi lựa chọn mô hình Lasso và tối ưu siêu tham số, nhóm tiến hành trích xuất trọng số của các đặc trưng để xác định mức độ quan trọng tương đối của từng biến đầu vào. Kết quả trực quan hoá các đặc trưng quan trọng được thể hiện trong Hình 6.



Hình 6: Các đặc trưng quan trọng theo 6 mô hình

Từ Hình 6, cả 6 mô hình đều đồng thuận rằng **Total Fat (g)** và **Carbs (g)** là hai đặc trưng quan trọng nhất ảnh hưởng đến lượng Calories của món ăn nhanh.

5 Bảng công việc